Project Title: Student Course Registration & Management System

---

1. Introduction

1.1 Purpose

The purpose of this mini project is to design and develop a **Servlet-based web application** that demonstrates the practical use of:

```
1. Servlet fundamentals and lifecycle methods
2. Sessions and cookies
3. RequestDispatcher
4. sendRedirect
5. JDBC database connection
6. CRUD operations
```

2. Project Overview

2.1 Project Name

```
Student Course Registration & Management System
```

2.2 Project Type

```
Java Web Application using Servlet, JSP, JDBC, HTML, CSS, and MySQL
```

2.3 Application Summary

The system allows an admin user to log in, manage student records, manage course records, and assign students to courses. The application should maintain login state using **sessions**, remember the admin username using **cookies**, display data using **JSP pages**, and perform all major data operations using **JDBC CRUD operations**.

3. Technologies Required

Layer	Technology
Frontend	HTML, CSS, JSP
Backend	Java Servlets
Server	Apache Tomcat
Database	MySQL
Database Connectivity	JDBC
IDE	Eclipse Enterprise Edition
Build Type	Dynamic Web Project or Maven Web Project

---

4. Main Learning Objectives

After completing this mini project, students should be able to:

Topic	Expected Learning
Servlet	Understand request handling through `doGet()` and `doPost()`
Servlet Lifecycle	Use and observe `init()`, `service()` / request methods, and `destroy()`
Session	Maintain login state across multiple requests
Cookie	Store small non-sensitive browser-side data
RequestDispatcher	Forward request data to JSP pages
sendRedirect	Redirect users after successful actions
JDBC	Connect Java web application with MySQL
CRUD	Insert, read, update, and delete records from database
MVC	Separate Servlet, JSP, DAO, and Model responsibilities

---

5. Scope of the Project

5.1 In Scope

The system should include:

```
1. Admin login
2. Remember username using cookies
3. Session-based dashboard access
4. Student CRUD
5. Course CRUD
6. Student-course registration
7. View registered students with course details
```

```
8. Logout
9. Basic validation
10. Proper use of RequestDispatcher and sendRedirect
11. Servlet lifecycle demonstration
```

5.2 Out of Scope

The following are not required for this mini project:

```
1. Online payment
2. Email notification
3. Password encryption
4. Spring MVC or Spring Boot
5. REST API
6. Advanced frontend frameworks
7. Role-based multiple user system
8. File upload
```

---

6. User Roles

6.1 Admin

The admin is the main user of the system.

Admin can:

```
1. Log in
2. View dashboard
3. Add student
4. View student list
5. Update student
6. Delete student
7. Add course
8. View course list
9. Update course
10. Delete course
11. Register student to course
12. View registrations
13. Logout
```

---

7. Main Modules

The project should be divided into the following modules:

```
1. Login Module
2. Dashboard Module
3. Student Management Module
4. Course Management Module
5. Student Course Registration Module
6. Session and Cookie Module
7. Logout Module
8. Database Connectivity Module
```

---

8. Functional Requirements

8.1 Login Module

Description

The system should allow the admin to log in using valid credentials.

Input Fields

Field	Description
Username	Admin username
Password	Admin password
Remember Username	Optional checkbox

Fixed Login Credentials

For this mini project, fixed credentials can be used:

Username	Password
admin	admin123

Functional Rules

```
1. If username and password are correct, create a session.
2. If “Remember Username” is selected, store username in a cookie.
3. If login is successful, redirect admin to dashboard.
4. If login fails, forward admin back to login page with error message.
5. Password should not be stored in cookie or session.
```

Servlet Concepts Used

Concept	Usage
Session	Store logged-in admin
Cookie	Remember username
RequestDispatcher	Show login error message
sendRedirect	Redirect to dashboard after successful login
Servlet Lifecycle	LoginServlet should include lifecycle method logging

---

8.2 Dashboard Module

Description

After login, the admin should see a dashboard.

Dashboard Should Display

```
1. Welcome message
2. Logged-in admin username
3. Total number of students
4. Total number of courses
5. Total number of registrations
6. Navigation links to all modules
```

Access Rule

```
Dashboard should be accessible only if the session exists.
If admin tries to access dashboard without login, redirect to login page.
```

Servlet Concepts Used

Concept	Usage
Session	Check whether admin is logged in
JDBC	Fetch dashboard count data
RequestDispatcher	Forward dashboard data to JSP
sendRedirect	Redirect unauthorized user to login page

8.3 Student Management Module

Description

Admin should be able to manage student records.

Student Fields

Field	Description
Student ID	Auto-generated unique ID
Student Name	Name of the student
Email	Student email
Phone	Student phone number
Age	Student age
City	Student city

Student CRUD Requirements

8.3.1 Add Student

The admin should be able to add a new student.

Validation rules:

Field	Rule
Student Name	Should not be empty
Email	Should not be empty
Phone	Should not be empty
Age	Should be greater than or equal to 18
City	Should not be empty

Expected flow:

```
Add Student Form
      ↓
AddStudentServlet
      ↓
If valid → insert into database → redirect to student list
If invalid → forward back to form with error message
```

Concepts used:

Concept	Usage
doGet	Display add student form
doPost	Process add student form
RequestDispatcher	Forward invalid data/error to form
sendRedirect	Redirect after successful insert
JDBC	Insert student record

8.3.2 View Students

The admin should be able to view all students.

Expected flow:

```
ViewStudentsServlet
      ↓
Fetch all students from database
      ↓
Forward student list to JSP
```

Concepts used:

Concept	Usage
JDBC	Fetch all student records
RequestDispatcher	Forward list to JSP
Session	Allow only logged-in admin

8.3.3 Update Student

The admin should be able to update student details.

Expected flow:

```
Student List
      ↓
Edit button
      ↓
EditStudentServlet
      ↓
Fetch existing student details
      ↓
Forward to update form
      ↓
Submit updated data
      ↓
UpdateStudentServlet
```

```
      ↓
Update database
      ↓
Redirect to student list
```

Concepts used:

Concept	Usage
doGet	Load existing student details
doPost	Process updated data
RequestDispatcher	Forward existing data to update JSP
sendRedirect	Redirect after successful update
JDBC	Fetch and update student record

8.3.4 Delete Student

The admin should be able to delete a student.

Expected flow:

```
Student List
      ↓
Delete button
      ↓
DeleteStudentServlet
      ↓
Delete student from database
      ↓
Redirect to student list
```

Concepts used:

Concept	Usage
JDBC	Delete student record
sendRedirect	Redirect after delete
Session	Protect delete operation

8.4 Course Management Module

Description

Admin should be able to manage courses.

Course Fields

Field	Description
Course ID	Auto-generated unique ID
Course Name	Name of course
Duration	Course duration
Fees	Course fees
Trainer Name	Trainer assigned to course

Course CRUD Requirements

8.4.1 Add Course

Validation rules:

Field	Rule
Course Name	Should not be empty
Duration	Should not be empty
Fees	Should be greater than 0
Trainer Name	Should not be empty

Expected flow:

```
Add Course Form
      ↓
AddCourseServlet
      ↓
If valid → insert course → redirect to course list
If invalid → forward back with error
```

8.4.2 View Courses

The system should show all available courses.

Expected display:

Course ID Course Name Duration Fees Trainer Name Actions

Actions:

```
Edit
Delete
```

8.4.3 Update Course

Admin should be able to edit course details.

Expected flow:

```
Course List
      ↓
Edit Course
      ↓
Load old course data
      ↓
Forward to update form
      ↓
Submit updated course
      ↓
Update database
      ↓
Redirect to course list
```

8.4.4 Delete Course

Admin should be able to delete a course if it is not actively assigned.

For mini project simplicity, delete can be allowed directly.

Expected flow:

```
Course List
      ↓
Delete Course
      ↓
Delete from database
      ↓
Redirect to course list
```

8.5 Student Course Registration Module

Description

Admin should be able to assign a student to a course.

Registration Fields

Field	Description
Registration ID	Auto-generated ID
Student	Selected student
Course	Selected course
Registration Date	Date of registration
Status	Active / Completed

Functional Requirements

```
1. Admin should see a registration form.
2. Student dropdown should show students from database.
3. Course dropdown should show courses from database.
4. Admin should register a student for a course.
5. System should save the registration in database.
6. Admin should view all registrations.
```

Expected flow:

```
Registration Form
      ↓
RegisterStudentCourseServlet
      ↓
Validate selected student and course
      ↓
Save registration in database
      ↓
Redirect to registration list
```

Concepts used:

Concept	Usage
JDBC	Fetch students, fetch courses, insert registration
RequestDispatcher	Forward student/course dropdown data to JSP
sendRedirect	Redirect after successful registration
Session	Protect registration pages

8.6 Logout Module

Description

Admin should be able to log out from the system.

Functional Rules

```
1. Logout should destroy the current session.
2. After logout, admin should be redirected to login page.
3. Dashboard should not be accessible after logout.
```

Concepts used:

Concept	Usage
Session	Invalidate session
sendRedirect	Redirect to login page

---

9. Servlet Lifecycle Requirements

The project must clearly demonstrate Servlet lifecycle.

At least the following Servlets should override lifecycle-related methods:

```
1. LoginServlet
2. DashboardServlet
3. StudentServlet / StudentControllerServlet
4. CourseServlet / CourseControllerServlet
```

Each Servlet should demonstrate:

Method	Expected Purpose
`init()`	Print initialization message or load Servlet-level configuration
`doGet()`	Handle page loading or fetch requests
`doPost()`	Handle form submission
`destroy()`	Print destruction message or cleanup message

Expected console observation:

```
LoginServlet initialized
Login request received
LoginServlet destroyed
```

Important: Students should understand that:

```
init() is called once when Servlet is loaded.
doGet() or doPost() is called for each request.
destroy() is called before Servlet is removed from service.
```

---

10. Session Requirements

The application must use sessions for login management.

10.1 Session Data

After successful login, store:

Session Attribute	Purpose
loggedInUser	Stores logged-in admin username
loginTime	Stores time of login, optional

10.2 Session Rules

```
1. Session should be created only after successful login.
2. Dashboard and CRUD pages should be accessible only if session exists.
3. If session does not exist, user should be redirected to login page.
4. Logout should invalidate the session.
```

10.3 Protected Pages

The following pages must be protected:

```
1. Dashboard
2. Student list
3. Add student
4. Update student
5. Delete student
6. Course list
7. Add course
8. Update course
9. Delete course
10. Registration list
```

---

11. Cookie Requirements

The application should use cookies for remembering the admin username.

11.1 Cookie Data

Cookie Name	Purpose
rememberedUsername	Stores admin username if remember option is selected

11.2 Cookie Rules

```
1. If admin selects “Remember Username”, username should be stored in cookie.
2. Password must never be stored in cookie.
3. If admin does not select remember option, username cookie should not be
created.
4. Existing username cookie should be deleted if remember option is
unchecked.
5. Login page should pre-fill or display remembered username if cookie
exists.
```

---

12. RequestDispatcher Requirements

`RequestDispatcher` must be used whenever valid or processed data needs to be sent from Servlet to JSP within the same request.

Use `RequestDispatcher` for:

```
1. Showing login error message
2. Forwarding dashboard data to dashboard.jsp
3. Forwarding student list to student-list.jsp
4. Forwarding course list to course-list.jsp
5. Forwarding existing student data to update-student.jsp
6. Forwarding existing course data to update-course.jsp
7. Forwarding validation errors back to form pages
8. Forwarding registration data to registration-list.jsp
```

Rule:

```
Use RequestDispatcher when request data must be preserved.
```

---

13. sendRedirect Requirements

`sendRedirect` must be used after successful actions where a fresh request is required.

Use `sendRedirect` for:

```
1. Login success → dashboard
2. Logout → login page
3. Add student success → student list
4. Update student success → student list
5. Delete student success → student list
6. Add course success → course list
7. Update course success → course list
8. Delete course success → course list
9. Student-course registration success → registration list
```

Rule:

```
Use sendRedirect after insert, update, delete, login success, and logout.
```

This follows the **Post-Redirect-Get** pattern and avoids duplicate form submission.

---

14. Database Requirements

14.1 Database Name

```
student_course_db
```

14.2 Tables Required

The system should contain four main tables:

```
1. admin
2. students
3. courses
4. registrations
```

14.3 Admin Table

Column	Type	Constraint	Description
admin_id	INT	Primary Key, Auto Increment	Unique admin ID
username	VARCHAR(50)	Not Null, Unique	Admin username
password	VARCHAR(100)	Not Null	Admin password

Sample record:

username	password
admin	admin123

14.4 Students Table

Column	Type	Constraint	Description
student_id	INT	Primary Key, Auto Increment	Unique student ID
student_name	VARCHAR(100)	Not Null	Student name
email	VARCHAR(100)	Not Null	Student email
phone	VARCHAR(15)	Not Null	Student phone
age	INT	Not Null	Student age
city	VARCHAR(50)	Not Null	Student city

14.5 Courses Table

Column	Type	Constraint	Description
course_id	INT	Primary Key, Auto Increment	Unique course ID
course_name	VARCHAR(100)	Not Null	Course name
duration	VARCHAR(50)	Not Null	Course duration
fees	DOUBLE	Not Null	Course fees
trainer_name	VARCHAR(100)	Not Null	Trainer name

14.6 Registrations Table

Column	Type	Constraint	Description
registration_id	INT	Primary Key, Auto Increment	Unique registration ID
student_id	INT	Foreign Key	Refers to students table
course_id	INT	Foreign Key	Refers to courses table
registration_date	DATE	Not Null	Date of registration
status	VARCHAR(20)	Not Null	Active / Completed

---

15. CRUD Operation Requirements

15.1 Student CRUD

Operation	Requirement
Create	Add new student
Read	View all students
Update	Edit existing student
Delete	Delete student

15.2 Course CRUD

Operation	Requirement
Create	Add new course
Read	View all courses
Update	Edit existing course
Delete	Delete course

15.3 Registration CRUD

For this mini project:

Operation	Requirement
Create	Register student to course
Read	View all registrations
Update	Update registration status
Delete	Cancel registration

---

16. Suggested Application Pages

16.1 HTML Pages

Page	Purpose
login.html / login.jsp	Admin login
index.html	Optional landing page

16.2 JSP Pages

JSP Page	Purpose
dashboard.jsp	Admin dashboard
student-form.jsp	Add student form
student-list.jsp	View students
update-student.jsp	Update student
course-form.jsp	Add course form
course-list.jsp	View courses
update-course.jsp	Update course
registration-form.jsp	Register student to course
registration-list.jsp	View registrations
error.jsp	Display general errors

---

17. Suggested Servlet List

Servlet	Responsibility
LoginServlet	Process login
LogoutServlet	Destroy session
DashboardServlet	Load dashboard
AddStudentServlet	Add student
ViewStudentsServlet	View students
EditStudentServlet	Load student for update
UpdateStudentServlet	Update student
DeleteStudentServlet	Delete student
AddCourseServlet	Add course
ViewCoursesServlet	View courses
EditCourseServlet	Load course for update
UpdateCourseServlet	Update course
DeleteCourseServlet	Delete course
RegistrationFormServlet	Load students and courses for registration form
RegisterStudentCourseServlet	Save registration

Servlet	Responsibility
ViewRegistrationsServlet	View registrations

---

18. Suggested Package Structure

```
src/main/java
│
└── com.studentcourse
    │
    ├── controller
    │   ├── LoginServlet.java
    │   ├── LogoutServlet.java
    │   ├── DashboardServlet.java
    │   ├── AddStudentServlet.java
    │   ├── ViewStudentsServlet.java
    │   ├── UpdateStudentServlet.java
    │   ├── DeleteStudentServlet.java
    │   ├── AddCourseServlet.java
    │   ├── ViewCoursesServlet.java
    │   ├── UpdateCourseServlet.java
    │   ├── DeleteCourseServlet.java
    │   ├── RegisterStudentCourseServlet.java
    │   └── ViewRegistrationsServlet.java
    │
    ├── dao
    │   ├── AdminDAO.java
    │   ├── StudentDAO.java
    │   ├── CourseDAO.java
    │   └── RegistrationDAO.java
    │
    ├── model
    │   ├── Admin.java
    │   ├── Student.java
    │   ├── Course.java
    │   └── Registration.java
    │
    └── util
        └── DBConnection.java
```

---

19. Suggested Web Folder Structure

```
src/main/webapp
│
├── login.jsp
├── index.html
│
├── css
│   └── style.css
│
```

```
└── WEB-INF
    │
    └── views
        ├── dashboard.jsp
        ├── student-form.jsp
        ├── student-list.jsp
        ├── update-student.jsp
        ├── course-form.jsp
        ├── course-list.jsp
        ├── update-course.jsp
        ├── registration-form.jsp
        ├── registration-list.jsp
        └── error.jsp
```

Important:

```
JSP pages inside WEB-INF should not be accessed directly from browser.
They should be accessed only through Servlets.
```

---

20. Data Flow Diagram Style Explanation

20.1 Login Flow

```
Login Page
   ↓
LoginServlet
   ↓
Validate credentials
   ↓
If valid:
   Create session
   Create cookie if remember username selected
   Redirect to dashboard

If invalid:
   Forward to login page with error message
```

20.2 Add Student Flow

```
Add Student Page
   ↓
AddStudentServlet
   ↓
Validate input
   ↓
If valid:
   StudentDAO inserts data into database
   Redirect to ViewStudentsServlet
```

```
If invalid:
   Forward back to student form with error
```

20.3 View Students Flow

```
ViewStudentsServlet
   ↓
StudentDAO fetches all students
   ↓
Set student list in request attribute
   ↓
Forward to student-list.jsp
```

20.4 Update Student Flow

```
Student List
   ↓
Edit Student
   ↓
EditStudentServlet
   ↓
Fetch selected student from database
   ↓
Forward to update-student.jsp
   ↓
Submit updated data
   ↓
UpdateStudentServlet
   ↓
Update database
   ↓
Redirect to student list
```

20.5 Delete Student Flow

```
Student List
   ↓
Delete Student
   ↓
DeleteStudentServlet
   ↓
Delete student from database
   ↓
Redirect to student list
```

---

21. Validation Requirements

21.1 Login Validation

Field	Validation
Username	Required
Password	Required

21.2 Student Validation

Field	Validation
Student Name	Required
Email	Required
Phone	Required
Age	Must be 18 or above
City	Required

21.3 Course Validation

Field	Validation
Course Name	Required
Duration	Required
Fees	Must be greater than 0
Trainer Name	Required

21.4 Registration Validation

Field	Validation
Student	Must be selected
Course	Must be selected
Status	Must be selected

---

22. Non-Functional Requirements

22.1 Usability

```
1. Pages should be simple and understandable.
2. Navigation should be clear.
3. Error messages should be meaningful.
4. Forms should have proper labels.
```

22.2 Security

```
1. Dashboard and CRUD pages should not open without login.
2. Password should not be stored in cookies.
3. Session should be destroyed on logout.
4. Direct access to JSP pages inside WEB-INF should not be allowed.
5. User input should be validated before database operation.
```

22.3 Maintainability

```
1. Servlet should handle request processing only.
2. DAO should handle database logic.
3. Model classes should represent data.
4. JSP should display output only.
5. Repeated code should be avoided where possible.
```

22.4 Performance

```
1. Database connection should be opened and closed properly.
2. Student and course lists should be fetched only when required.
3. Large data should not be stored in session.
```

---

23. Mandatory Concept Mapping

Required Topic	Where It Should Be Used
Servlet lifecycle	LoginServlet, DashboardServlet, StudentServlet
Session	Login, dashboard protection, logout
Cookie	Remember username
RequestDispatcher	Forward data/errors to JSP
sendRedirect	After login success, logout, CRUD success
JDBC connection	DAO classes
Create operation	Add student, add course, register student
Read operation	View students, courses, registrations

Required Topic	Where It Should Be Used
Update operation	Edit student, edit course, update registration status
Delete operation	Delete student, course, registration

---

24. Expected Output Screens

The final project should include the following working screens:

```
1. Login Page
2. Dashboard Page
3. Add Student Page
4. Student List Page
5. Update Student Page
6. Add Course Page
7. Course List Page
8. Update Course Page
9. Student Course Registration Page
10. Registration List Page
```

---

25. Error Handling Requirements

The system should handle:

```
1. Invalid login
2. Empty form fields
3. Invalid age
4. Invalid fees
5. Invalid student ID
6. Invalid course ID
7. Database connection failure
8. Record not found
9. Unauthorized access without session
```

For validation errors:

```
Use RequestDispatcher to forward back to the form with error message.
```

For successful database actions:

```
Use sendRedirect to redirect to list page.
```

---

26. Database Connection Requirement

The project should have a reusable database connection utility.

The database utility should be responsible for:

```
1. Loading database driver
2. Creating connection
3. Returning connection to DAO classes
4. Handling database connection exceptions
```

The DAO classes should be responsible for:

```
1. Insert operations
2. Select operations
3. Update operations
4. Delete operations
```

Servlets should not directly contain SQL queries in a well-structured submission.

---

27. Project Constraints

```
1. The project must run on Apache Tomcat.
2. The project must use Servlet and JSP.
3. Database operations must be done using JDBC.
4. No Spring Boot or Hibernate should be used.
5. Session must be used for login state.
6. Cookie must be used for remember username.
7. RequestDispatcher and sendRedirect must both be used properly.
8. CRUD operations must be implemented using database.
```

---

28. Acceptance Criteria

The project will be accepted only if:

```
1. Admin can log in successfully.
2. Invalid login shows proper error.
3. Session is created after login.
4. Dashboard is not accessible without login.
5. Remember username cookie works.
6. Admin can add, view, update, and delete students.
7. Admin can add, view, update, and delete courses.
8. Admin can register students to courses.
9. Registration records can be viewed.
10. Logout destroys the session.
```

11. Valid data is forwarded using RequestDispatcher where required.
12. Successful actions use sendRedirect.
13. Database connection works properly.
14. Servlet lifecycle methods are demonstrated.